# Data Governance Copilot

Interview Question Bank

Topic 01 — LangGraph & Agent Orchestration

12 questions · 4 sections · Difficulty: Foundational → Gotcha

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

## Core Concepts

**Q01** Foundational

### What is LangGraph and how does it differ from a simple LangChain chain?

*Hint: Think about statefulness and graph topology vs linear chains.*

**Key points to cover:**

- LangGraph builds on LangChain to support stateful, cyclical computation graphs (not just linear pipelines)
- Key abstraction: StateGraph — nodes are Python functions, edges define transitions
- Supports conditional edges (branching), loops, and checkpointing out-of-the-box
- Your project uses StateGraph with sequential routing to avoid InvalidUpdateError on non-Annotated keys
- Contrast: LangChain LCEL is DAG/linear; LangGraph allows cycles and persistent state

Q02 Foundational

### Explain the AgentState TypedDict in your project. Why is total=False important?

*Hint: Think about partial state updates across nodes.*

**Key points to cover:**

- total=False means all keys are optional — nodes only write the fields they touch
- Allows partial state updates: supervisor writes intent/next_agents; agents write agent_results
- Without total=False, every node would need to supply all fields or cause TypedDict validation errors
- Annotated fields (agent_results, sources etc.) use operator.add for list accumulation across nodes
- Non-Annotated fields (intent, final_summary etc.) are last-write-wins — reason sequential routing was chosen

Q03 Intermediate

### Walk me through what happens from the moment a query hits your system to the final_summary.

*Hint: Trace the full graph flow step by step.*

**Key points to cover:**

- 1. pre_hook: guardrails check (length, SQL injection, prompt injection, PII redaction), sets start_time & query_id
- 2. supervisor_node: calls LLM for intent classification → writes intent, next_agents to state
- 3. _agent_router (conditional edge): picks first agent from next_agents[]
- 4. Agent node (e.g. information_node): executes with @cached_node + retry_agent_call, appends to agent_results
- 5. _after_agent (conditional): loops through remaining agents via _agents_ran tracker
- 6. auto_ticket_node: HITL gate — first pass sets pending_action; second pass (approved=True) creates Jira tickets
- 7. synthesizer_node: combines all agent_results into final_summary via LiteLLM
- 8. post_hook: calculates execution_ms

## Sequential Routing & the InvalidUpdateError Fix

Q04 Advanced

### What is the InvalidUpdateError in LangGraph and why did parallel fan-out cause it in your project?

*Hint: Think about concurrent writes to the same state key.*

**Key points to cover:**

- LangGraph's parallel fan-out runs multiple nodes simultaneously — each writes to shared AgentState
- Non-Annotated keys (like intent, final_summary) have no merge strategy — last write wins, but concurrent writes race
- LangGraph raises InvalidUpdateError when two nodes try to update the same non-Annotated key in the same step
- Your agents all write to agent_results but also touch shared keys — parallel execution broke this
- Fix: switch to sequential routing so only one agent runs per graph step, eliminating concurrent writes

Q05 Advanced

### Explain the _agents_ran list pattern. How does _wrap_agent work and why was it necessary?

*Hint: This was a custom solution to LangGraph's routing limitation.*

**Key points to cover:**

- _agents_ran: List[str] tracks which agents have already executed in this invocation
- _wrap_agent(agent_fn) wraps each agent node — after execution it appends agent name to _agents_ran
- _after_agent conditional edge checks _agents_ran vs next_agents to find the next unrun agent
- This creates a sequential loop without LangGraph cycles — each conditional edge re-evaluates the tracker
- Avoids the need for a true loop node; works within LangGraph's DAG model while simulating sequential iteration
- Critical: _agents_ran must be in AgentState TypedDict (Bug #44 — was missing initially)

```
# Conceptual pattern def _after_agent(state): ran = set(state.get('_agents_ran', [])) for agent in state['next_agents']: if agent not in ran: return agent # route to this node return 'auto_ticket' # all done
```

Q06 Intermediate

### Why are Annotated fields with operator.add used for agent_results but not for intent or final_summary?

*Hint: Think about what each field represents semantically.*

**Key points to cover:**

- agent_results accumulates contributions from multiple agents — needs operator.add to merge lists across nodes
- sources, anomalies, errors similarly accumulate across agents — Annotated[List, operator.add]
- intent is singular — only supervisor writes it; last-write-wins is correct semantics
- final_summary is singular — only synthesizer writes it; no accumulation needed
- Using operator.add on intent would concatenate strings from every node that touches state — wrong behavior
- Rule of thumb: Annotated + operator.add for fields written by multiple nodes; plain type for single-writer fields

## Nodes, Edges & Graph Singleton

Q07 Intermediate

### What is the _graph singleton pattern in graph.py and why does it matter for production?

*Hint: Consider ECS multi-task deployments.*

**Key points to cover:**

- _graph is a module-level variable initialized once via get_graph() — subsequent calls return cached instance
- Prevents re-compiling the StateGraph on every request — compilation is expensive (validates edges, nodes)
- copilot_graph is a proxy object for backward compatibility — delegates to the singleton
- In ECS Fargate, each task has its own process — singleton is per-process, not global
- For tests: patch get_graph at module level to inject a mock graph without re-importing
- Thread safety: LangGraph graph objects are stateless after compilation — safe to share across coroutines

**Q08** Intermediate

### Explain the pre_hook and post_hook nodes. What do they guard and measure?

*Hint: These are cross-cutting concerns separated from business logic.*

**Key points to cover:**

- pre_hook: runs before supervisor — guardrails (length/SQL/injection/PII), sets start_time, generates query_id
- guardrail_passed=False routes directly to post_hook, skipping all agents (short-circuit)
- guardrail_reason written to state for downstream audit logging
- post_hook: calculates execution_ms = (now - start_time) * 1000, writes to state
- Separation of concerns: business agents don't need to know about timing or security checks
- query_id enables distributed tracing — correlates LangSmith traces with API logs

---

**Q09** Advanced

### How would you add a new agent to the graph? Walk through every file you'd need to touch.

*Hint: This tests understanding of the full graph wiring.*

**Key points to cover:**

- 1. agents/new_agent.py — implement BaseAgent with execute() returning AgentResult
- 2. services/ — add real + mock service if the agent needs external data
- 3. graph/routing.py — add 'new_agent' to relevant INTENT_AGENT_MAP entries
- 4. graph/nodes.py — add new_node() function with @cached_node if appropriate, wire retry
- 5. graph/graph.py — add_node('new_agent', _wrap_agent(new_node)), add conditional edges to router
- 6. graph/state.py — add any new state fields the agent writes
- 7. tests/ — add test_new_agent.py with injection pattern (mock service)

---

## Tricky / Gotcha Questions

**Q10** Gotcha

### If you forgot to add _agents_ran to the AgentState TypedDict, what would happen at runtime?

*Hint: This was actual Bug #44 in your project.*

**Key points to cover:**

- Python TypedDict with total=False won't raise an error on missing keys at definition time
- The bug manifests at runtime: state.get('_agents_ran', []) returns [] but state['_agents_ran'] = [...] silently drops the write
- LangGraph only persists keys declared in the TypedDict — undeclared keys are discarded
- Result: _agents_ran is always empty after each node, so _after_agent always routes to the first agent again
- Causes infinite loop: same agent runs repeatedly until LangGraph's recursion limit is hit
- Fix: declare _agents_ran: List[str] in AgentState (Bug #44 fix)

---

**Q11** Gotcha

### Can you have two synthesizer nodes in the graph? What would happen?

*Hint: Think about state key collisions.*

**Key points to cover:**

- Technically yes — LangGraph allows multiple nodes writing to the same key if sequential
- But final_summary is not Annotated — second synthesizer would overwrite the first (last-write-wins)
- If run in parallel, InvalidUpdateError would be raised
- Better pattern: pass both summaries as a list via an Annotated field, then have a final merger node
- In practice: one synthesizer is sufficient — it receives all agent_results and composes a single summary

---

**Q12** Design

## Your graph uses LangSmith for observability. In production you want to switch to CloudWatch/X-Ray. What changes?

*Hint: AWS-native observability migration.*

**Key points to cover:**

- Remove LANGSMITH_TRACING=true and LANGSMITH_API_KEY from prod env vars
- Add AWS X-Ray SDK: instrument LangGraph nodes with xray_recorder.capture() decorator
- Route structured logs (already JSON via logging_utils.py) to CloudWatch Logs via awslogs driver in ECS task definition
- Add custom CloudWatch metrics for execution_ms, confidence score, intent distribution
- LangGraph supports custom callbacks — implement a CloudWatchCallbackHandler to capture token usage
- Advantage: no data egress to LangSmith SaaS; stays within AWS VPC; integrated with ECS alarms